

ifcc — Mini-compileur C

Compilateur d'un sous-ensemble de C, écrit en C++ avec ANTLR4

Projet PLD-COMP — INSA Lyon

Équipe : Luka Maret · Valentin Dury · Sarah Ripoche · Adina Valkova · Louis Zyzelewicz · Clément Dubois

Idée globale

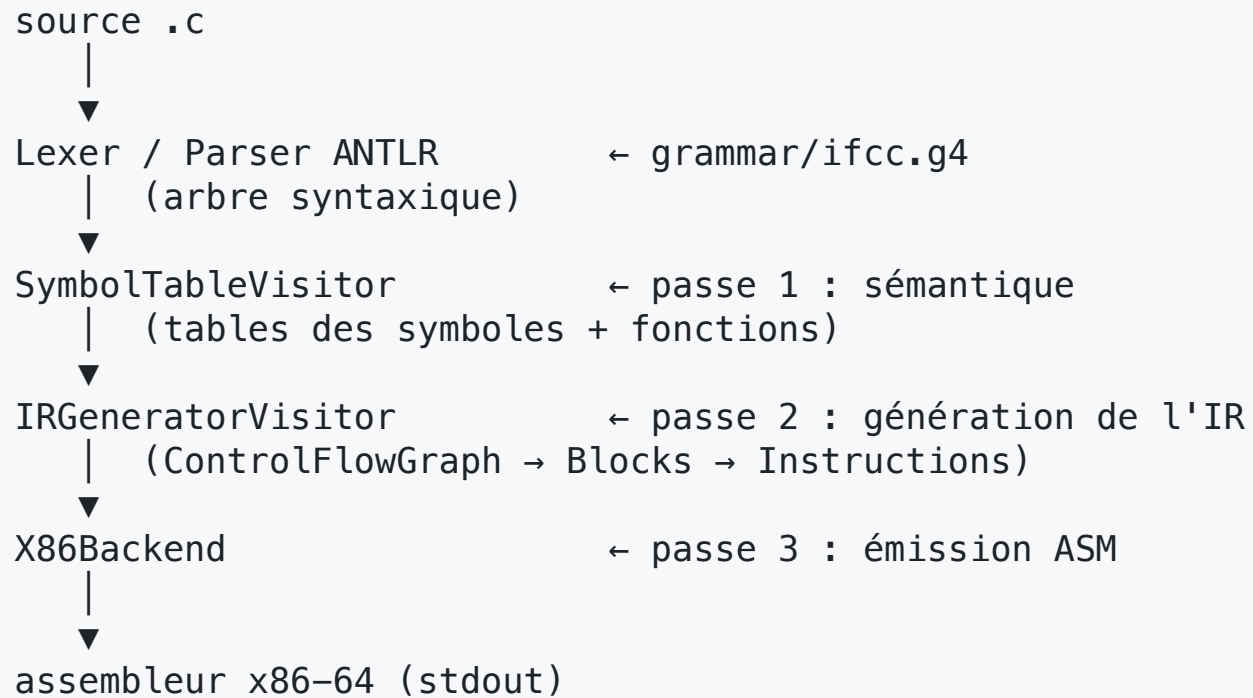
`ifcc` prend un fichier `.c` (sous-ensemble du langage) et produit de l'**assembleur x86-64 AT&T** sur la sortie standard.

Le critère de réussite est différentiel : un test passe quand `ifcc` et **GCC** **sont d'accord** — soit les deux acceptent le programme et renvoient le même code de sortie, soit les deux le rejettent.

```
./build/ifcc tests/cases/1_return_literal.c    # → assembleur sur stdout  
gcc fichier.s -o prog && ./prog ; echo $?      # exécute le résultat
```

Philosophie : un vrai pipeline de compilation à petite échelle, avec une séparation nette **front-end** / **IR** / **back-end**.

Le pipeline en une image



Trois passes orchestrées par `main.cpp` .

Organisation du dépôt

grammar/ifcc.g4	Grammaire ANTLR (lexer + parser)
generated/	Code ANTLR généré au build (NE PAS éditer)
include/ + src/	Code C++ du compilateur
├─ SymbolTableVisitor	passe 1
├─ IRGeneratorVisitor	passe 2
├─ ControlFlowGraph / Block / Instruction	structure de l'IR
├─ instructions/	une classe par instruction IR
├─ backend/	Backend (abstrait), X86Backend, ARMBackend
└─ utils/	Type, Variable
tests/cases/	58 programmes .c de test
tests/ifcc-test.py	runner : compare ifcc vs GCC
config/	chemins ANTLR par plateforme (macOS, linux, IF501...)
demo/	scripts de démonstration
Makefile / Dockerfile	build natif ou Docker

Outils & build

- **Langage** : C++17. **Génération parser** : ANTLR 4.13.2 (Java au build).
- Le **Makefile** **auto-détecte la plateforme** (`uname -m`) et inclut le bon fichier `config/config-*.mk` (`ANTLRJAR` , `ANTLRINC` , `ANTLRLIB`).
- Les `.cpp` de `src/` sont **découverts automatiquement** (pas de liste à maintenir).

```
make          # build → build/ifcc
make test     # build + tous les tests de tests/cases/
make clean    # supprime build/, generated/, ifcc-test-output/
make gui FILE=... # visualise l'arbre syntaxique (Java)
make asm FILE=... # affiche l'IR (--debug-ir) + l'assembleur

make docker / docker-test / docker-build  # même chose, isolé dans Docker
```

Docker garantit la reproductibilité entre les machines de l'équipe.

Passé 1 — SymbolTableVisitor

Premier parcours de l'arbre. Rôle : **construire les tables de symboles et vérifier la sémantique** avant toute génération de code.

- Une table `map<string, Variable>` **par fonction** (`allSymbolTables`).
- Une table des fonctions (`functionTable`) : type de retour + types des paramètres.
- Contrôles effectués (arrêt avec message d'erreur si violation) :
 - variable **déjà déclarée** → erreur de redéfinition ;
 - variable **utilisée sans déclaration** → erreur ;
 - variable déclarée `void` → erreur ;
 - marquage `used` des variables (variables non utilisées repérables).
- `main.cpp` vérifie ensuite qu'une fonction `main` existe.

Passé 2 — IRGeneratorVisitor (1/2)

Deuxième parcours : transforme l'arbre en **représentation intermédiaire** (IR).

L'IR est un **graphe de flot de contrôle** (CFG) :

```
ControlFlowGraph  -owns→ [Block, Block, ...]  
Block             -owns→ [Instruction, Instruction, ...]  
                  + true_case_block / false_case_block (arêtes du CFG)
```

- Chaque `visitXxx` renvoie le **nom de la variable** (ou temporaire) contenant le résultat de la sous-expression → composition naturelle des expressions.
- Les temporaires `$temp0`, `$temp1...` sont alloués par `CFG::addTempVariable`.
- La variable spéciale `$return` porte la valeur de retour (chargée dans `%eax` à l'épilogue).

Passe 2 — IRGeneratorVisitor (2/2) : optimisations

Optimisations faites à la volée, pendant la génération de l'IR :

Piage de constantes (`knownConstants`) — toute opération entre deux constantes connues est évaluée à la compilation :

```
2 + 3 * 4 → une seule constante 14
```

Simplifications algébriques — éléments neutres / absorbants éliminés :

```
x + 0, x - 0, x * 1, x / 1, x | 0 → x  
x * 0, x & 0 → 0
```

Code mort — après un `return`, un bloc « dead » est créé ; le nombre de blocs inatteignables est compté et signalé (`warning: N unreachable block(s)`).
Division/modulo par zéro **non** pliés (laissés au runtime).

La structure de l'IR — types & variables

utils/Type.h — types supportés et règles :

```
enum class Type { VOID, CHAR, INT32, DOUBLE };  
promote(left, right) // double domine, sinon int (char promu en int)  
typeSize: char=4, int=4, double=8, void=0
```

utils/Variable.h — une variable de l'IR :

```
struct Variable {  
    Type type;  
    int offset; // emplacement sur la pile  
    int pointerDepth; // nombre d'étoiles (int** → 2)  
    bool used;  
    int size() const; // pointeur → 8 octets, sinon typeSize(type)  
};
```

size() pilote le choix `movl` (4 o) vs `movq` (8 o) côté backend.

Passe 3 — X86Backend & double-dispatch

Le backend traduit chaque instruction IR en assembleur. Mécanisme clé : **double-dispatch** (pattern Visitor inversé) :

```
// Block::generateASM appelle pour chaque instruction :  
instruction->generate(backend, output);  
// Add::generate fait simplement :  
void Add::generate(Backend &b, ostream &o) { b.emit(this, o); }  
// → résolution sur le type concret Add : X86Backend::emit(Add*)
```

→ L'IR ignore tout de l'assembleur ; ajouter une **cible** = nouvelle classe

Backend . ARMBackend (AArch64 / Apple Silicon) implémente la même interface.

Dans cette version, main.cpp instancie en dur X86Backend .

Backend — gestion de la pile (System V)

- **Prologue** : `pushq %rbp ; movq %rsp, %rbp` , puis copie des paramètres (registres `%rdi, %rsi, %rdx, %rcx, %r8, %r9` pour les 6 premiers ; pile au-delà), puis réservation du cadre **aligné sur 16 octets** (exigence ABI).
- **Variables locales** : adressées `-N(%rbp)` (`varToLocation` nie l'offset positif du CFG).
- **Épilogue** : `movl $return, %eax ; leave ; ret` .
- **Appels** (`CallFunction`) : args 7+ empilés en ordre inverse, padding pour garder `%rsp` aligné, nettoyage de la pile après `call` .

<code>addl/subl/imull</code>	<code>+ - *</code>
<code>cdq ; idivl</code>	<code>/ et %</code> (<code>movl %eax</code> pour <code>/</code> , <code>movl %edx</code> pour <code>%</code>)
<code>andl/orl/xorl</code>	<code>& ^</code>
<code>cmpl ; setX ; movzbl</code>	comparaisons (<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>==</code> , <code>!=</code>)

Concepts implémentés — le langage

- **Types** : `int` , `char` , `void` (+ `double` partiellement dans les types).
- **Expressions** : `+` `-` `*` `/` `%` , unaire `-` , parenthèses, **précédence** correcte.
- **Opérateurs bit-à-bit** : `&` `|` `^` avec leur précédence.
- **Comparaisons** : `<` `>` `<=` `>=` `==` `!=` .
- **Variables** : déclarations (multiples, avec/sans init), affectations chaînées.
- **Caractères** : littéraux `'A'` .
- **Fonctions** : définition, paramètres (avec passage pile au-delà de 6), appels, récursion, `putchar` / `getchar` .
- **Contrôle de flux** : `if` / `else` / `else if` , `while` (imbriqués).
- **Pointeurs** : `&x` , `*p` , `**pp` , écriture `*p = ...` , paramètres pointeurs.

Exemple — grammaire des expressions

L'ordre des alternatives dans `ifcc.g4` encode la **précédence** (la plus prioritaire en haut) ; ANTLR gère l'associativité à gauche automatiquement :

```
expression
: op=(MINUS | NOT | BITWISE_AND | TIMES) expression # unary_operation
| expression op=(TIMES | DIVIDE | MODULO) expression # multiplicative_expression
| expression op=(PLUS | MINUS) expression           # additive_expression
| expression op=(LESSER | ... | GREATER) expression # comparison_expression
| expression op=(COMPARE_EQUAL | NOT_EQUAL) expression # equal_expression
| expression BITWISE_AND expression                # bitwise_and_expression
| expression BITWISE_XOR expression                # bitwise_xor_expression
| expression BITWISE_OR expression                 # bitwise_or_expression
| BRACKET_OPEN expression BRACKET_CLOSE             # bracketed_expression
| CONSTANT                                           # constant_expression
| IDENTIFIER                                         # variable_expression
| IDENTIFIER '(' (expression (',' expression)*)? ')' # function_call
;
```

Chaque `# label` engendre un `visitLabel(...)` à surcharger.

Exemple — anatomie d'une instruction IR

Une instruction = une classe. Exemple `Add` (`include/instructions/Add.h`) :

```
class Add : public Instruction {
public:
    Add(Block *block, Type type, string dest, string left, string right)
        : Instruction(block, type), destination(dest), left(left), right(right) {}

    void generate(Backend &b, ostream &o) override { b.emit(this, o); } // dispatch
    void debug(ostream &o) const override {
        o << "  Add  " << destination << " = " << left << " + " << right << "\n"; }

    string getDestination() const { return destination; }
    string getLeft() const { return left; }
    string getRight() const { return right; }
private:
    string destination, left, right;
};
```

21 classes sur ce modèle : `Add`, `Subtract`, `Multiply`, `Divide`, `Modulo`, `Copy`, `LoadConstant`, `Negate`, `BitwiseAnd/Or/Xor`, `Equal`, `NotEqual`, `Lesser`, `Greater`, `LesserOrEqual`, `GreaterOrEqual`, `CallFunction`, `Reference`, `DereferenceRead/Write`.

Exemple — un `visitXxx` qui génère l'IR

`visitAdditive_expression` : visite les deux opérandes, tente le pliage, sinon émet une instruction et **renvoie le nom du résultat** :

```
antlrcpp::Any IRGeneratorVisitor::visitAdditive_expression(ctx) {
    string left  = asString(visit(ctx->expression(0))); // récursion
    string right = asString(visit(ctx->expression(1)));
    Type type = promote(getVar(left).type, getVar(right).type);
    bool isAddition = ctx->op->getType() == ifccParser::PLUS;

    int l, r;
    if (isConstant(left, l) && isConstant(right, r)) // pliage
        return emitConstant(type, isAddition ? l + r : l - r);
    if (isConstant(right, r) && r == 0) return left; // x + 0 → x

    string tmp = currentCFG->addTempVariable(type);
    Block *b = currentCFG->getCurrentBlock();
    b->addInstruction(isAddition ? (Instruction*)new Add(b, type, tmp, left, right)
                        : new Subtract(b, type, tmp, left, right));
    return tmp; // nom du temporaire
}
```

Exemple — émission assembleur (backend)

Côté `X86Backend`, chaque `emit(...)` écrit l'AT&T. Toutes les opérandes vivent sur la pile, donc on charge dans `%eax`, on calcule, on range :

```
void X86Backend::emit(Add *instr, ostream &o) {
    auto *cfg = instr->getBlock()->getControlFlowGraph();
    o << "    movl " << varToLocation(instr->getLeft(), cfg) << ", %eax\n";
    o << "    addl " << varToLocation(instr->getRight(), cfg) << ", %eax\n";
    o << "    movl %eax, " << varToLocation(instr->getDestination(), cfg) << "\n";
}
```

Division / modulo — un seul `idivl`, le quotient est dans `%eax`, le reste dans `%edx` :

```
void X86Backend::emit(Modulo *instr, ostream &o) {
    o << "    movl " << ...left... << ", %eax\n";
    o << "    cdq\n"; // étend le signe dans %edx
    o << "    idivl " << ...right... << "\n";
    o << "    movl %edx, " << ...dest... << "\n"; // reste pour le modulo
}
```


Exemple — les comparaisons

`a < b` produit **0 ou 1** via les instructions `setX` du x86 :

```
int a = 3; int b = 5;  
return a < b;           // → code de sortie 1
```

```
movl -4(%rbp), %eax      # a  
cmpl -8(%rbp), %eax      # compare a et b  
setl %al                 # %al = 1 si a < b, sinon 0  
movzbl %al, %eax         # étend l'octet sur 32 bits (zero-extend)  
movl %eax, -12(%rbp)     # résultat
```

Opérateur	setX	Opérateur	setX
<	setl	>=	setge
>	setg	==	sete
<=	setle	!=	setne

Exemple — fonctions & appels

```
int add(int a, int b) { return a + b; }  
int main() { return add(3, 4); }
```

Conventions System V appliquées par le backend :

```
add:  
    pushq %rbp ; movq %rsp, %rbp  
    movl %edi, -8(%rbp)      # a ← 1er arg (registre %edi)  
    movl %esi, -12(%rbp)    # b ← 2e  arg (registre %esi)  
    ...  
main:  
    movl $3, %edi           # 1er argument  
    movl $4, %esi           # 2e  argument  
    call add  
    movl %eax, -8(%rbp)     # récupère la valeur de retour
```

6 premiers args : %edi %esi %edx %ecx %r8d %r9d . Au-delà : sur la pile
(offsets positifs 16(%rbp) , 24(%rbp) ...), avec padding pour rester aligné.

Exemple — pointeurs en assembleur

```
int a = 10;
int *p = &a;    // p reçoit l'adresse de a
*p = 42;        // écrit 42 à l'adresse pointée → a vaut 42
```

```
movl $10, -8(%rbp)    # a = 10
leaq -8(%rbp), %rax    # &a (Reference → leaq)
movq %rax, -16(%rbp)  # p = &a (8 octets : movq !)

movl $42, %eax        # valeur 42
movq -16(%rbp), %rcx   # charge le pointeur p
movl %eax, (%rcx)      # écrit *p (DereferenceWrite)
```

Dès qu'un pointeur est impliqué, le backend passe en **movq (8 octets)** pour ne pas tronquer l'adresse. **&** → **leaq**, ***** (lecture) → **movl (%rax)**.

Focus : le contrôle de flux (**if** / **while**)

`visitStatement` construit les arêtes du CFG entre blocs. Pour un `if` :

```
[bloc condition]
├ false → [after_if]      (setFalseCaseBlock)
└ vrai  → [if_true] → [after_if]
                (+ [if_false] si else)
```

Émission dans `Block::generateASM` :

```
if (true_case_block)      emitJump(true_case_block);      // jmp
else if (false_case_block) emitFalseJump(false_case_block); // cmpl $0,%eax ; je
```

Le `while` relie `body` → `test` (retour de boucle) et `test` → `after` (sortie).

Exemple — une boucle `while` compilée

```
int main() {  
    int s = 0; int i = 0;  
    while (i < 5) { s = s + i; i = i + 1; }  
    return s;           // → 0+1+2+3+4 = 10  
}
```

Structure de blocs et sauts générés :

```
main_while_test:  
    movl -.., %eax ; cmpl $5, %eax ; setl %al ; movzbl %al, %eax  
    cmpl $0, %eax  
    je main_while_end      # condition fausse → on sort  
main_while_body:  
    ... s = s + i ; i = i + 1 ...  
    jmp main_while_test    # retour en tête de boucle  
main_while_end:  
    ...
```

Focus : les pointeurs

Le `pointerDepth` (nombre d'étoiles) suit chaque variable et temporaire.

- `&x` → instruction `Reference` (`leaq`), résultat de profondeur **+1**.
- `*p` (lecture) → `DereferenceRead` (`movq (%rax)`), profondeur **-1**.
- `*p = v` (écriture) → `DereferenceWrite` (écrit à `(%rcx)`).
- `**pp` → `evalAddress` enchaîne **n-1 déréférencements** pour n étoiles.

Le backend choisit `movq` (8 o) dès qu'un pointeur est impliqué, pour ne pas tronquer une adresse à 4 octets.

```
int a = 10; int *p = &a; *p = 42;    // a vaut 42
int **pp = &p; **pp = 7;             // a vaut 7
```

Pour `**pp = 7`, `evalAddress` enchaîne les déréférencements :

```
movq -16(%rbp), %rax    # charge pp
movq (%rax), %rax       # *pp (1 déréférencement intermédiaire) → adresse de a
movl $7, %ecx
movl %ecx, (%rax)       # écrit 7 à l'adresse finale
```

Exemple — optimisations à la compilation

Code source verbeux → IR minimal grâce au **pliage** et aux **simplifications** :

```
int x = 2 + 3 * 4;    // → x = 14      (tout plié)
int y = a * 0;        // → y = 0      (élément absorbant)
int z = a + 0;        // → z = a      (élément neutre, aucune instr)
int w = a * 1;        // → w = a
```

Détection de **code mort** après un `return` :

```
int main() {
    return 42;
    int x = 99;    // inatteignable
}
// stderr → warning: 1 unreachable code block(s) detected
```

Garde-fou : `a / 0` et `a % 0` ne sont **pas** pliés (laissés au runtime).

Exemple — erreurs détectées (passe 1)

`ifcc` rejette les programmes invalides, comme GCC. Quelques cas testés :

```
int main() { int a; int a; }      // Error: 'a' is already declared
int main() { return b; }          // Error: 'b' is not declared
int main() { void v; }            // Error: variable 'v' declared void
int main() { return f(); }        // Error: fonction non définie
int f(int x){return x;}
int main() { return f(1, 2); }    // Error: mauvais nombre d'arguments
int f() { return 0; }             // Error: no 'main' function defined
```

Chaque erreur affiche un message sur `stderr` et termine avec un code $\neq 0$.

Le test passe car **GCC rejette aussi** ces programmes.

Exemple bout-en-bout (1/3) — source & arbre

Source :

```
int main() {  
    int x = 2 + 3 * 4;  
    return x;  
}
```

Arbre syntaxique (simplifié) produit par ANTLR :

```
function "main"  
└─ block  
    └─ variable_declaration: int x = ( 2 + ( 3 * 4 ) )  
        additive  
            /      \  
        constant 2  multiplicative  
                    /      \  
                constant 3  constant 4  
    └─ return_statement: return x
```

La précedence (* avant +) est encodée par l'**ordre des règles** dans la grammaire.

Exemple bout-en-bout (2/3) — IR

IRGeneratorVisitor parcourt l'arbre. Grâce au **pliage de constantes**,

$3 * 4 = 12$ puis $2 + 12 = 14$ sont calculés à la compilation :

```
=== CFG: main ===  
[Block: main_entry]  
  LoadConstant $return = 0      ← init implicite du retour  
  LoadConstant $temp0 = 14      ← 2 + 3*4 plié en une constante  
  Copy          x       = $temp0  
  Copy          $return = x      → saut vers main_exit  
[Block: main_exit]
```

Sans pliage, on aurait eu un `LoadConstant 3`, un `LoadConstant 4`, un `Multiply`, un `LoadConstant 2`, un `Add` ... L'IR est ici **réduit à l'essentiel**.

On peut observer cet IR avec `./build/ifcc --debug-ir fichier.c`.

Exemple bout-en-bout (3/3) — assembleur

X86Backend traduit chaque instruction de l'IR :

```
.globl main
main:
    pushq %rbp
    movq %rsp, %rbp
    subq $16, %rsp          # cadre de pile aligné sur 16
    movl $0, -4(%rbp)       # $return = 0
    movl $14, -8(%rbp)      # $temp0 = 14
    movl -8(%rbp), %eax      # x = $temp0
    movl %eax, -12(%rbp)
    movl -12(%rbp), %eax     # $return = x
    movl %eax, -4(%rbp)
main_exit:
    movl -4(%rbp), %eax      # valeur de retour dans %eax
    leave
    ret
```

gcc sortie.s -o prog && ./prog ; echo \$? → 14.

Tests

- **58 cas** dans `tests/cases/`, numérotés par fonctionnalité croissante (arithmétique → variables → bitwise → fonctions → comparaisons → if/while → pointeurs).
- Inclut des cas qui **doivent échouer** : redéfinition, fonction non définie, mauvais nombre d'arguments, absence de `main`, syntaxe invalide.
- `tests/ifcc-test.py` compile chaque `.c` avec **GCC** et **ifcc**, exécute les deux binaires et **compare les codes de sortie**.
- `tests/ifcc-pretty.py` formate le rapport.

```
make test    # python3 tests/ifcc-test.py tests/cases | ifcc-pretty.py
```

Étendre le compilateur — recette

Pour ajouter une fonctionnalité du langage :

1. **Grammaire** : étendre `grammar/ifcc.g4` (nouvelle règle / token).
2. **Sémantique** : ajouter le `visitXxx` dans `SymbolTableVisitor` si besoin.
3. **IR** : ajouter le `visitXxx` dans `IRGeneratorVisitor` ;
créer une classe `Instruction` dans `include/instructions/` + `src/instructions/`.
4. **Backend** : ajouter la surcharge `emit(NouvelleInstr*)` dans `Backend`
(abstrait) puis dans `X86Backend` (et `ARMBackend`).

Le `Makefile` détecte les nouveaux `.cpp` automatiquement.

Points de conception à retenir

- **Séparation stricte** front-end (ANTLR) / IR (CFG) / back-end (ASM).
- **Double-dispatch** pour découpler instructions et cibles assembleur.
- IR sous forme de **CFG explicite** (blocs + arêtes vrai/faux), naturel pour le contrôle de flux et extensible aux optimisations.
- Optimisations simples mais réelles : **pliage de constantes**, simplifications algébriques, détection de **code mort**.
- **Portabilité** assurée par Docker + configs par plateforme.

Merci !